

A Controlled Comparative Study of Reinforcement Learning Algorithms for Dynamic Obstacle-Aware TurtleBot3 Navigation

Mathiyalagan Vasantharaj and Isaac Vivin Moses

*M.Sc. Robotics (Intelligent Robotics), Deggendorf Institute of Technology, Campus Cham
Cham, Germany*

Email: mathiyalagan.vasantharaj@stud.th-deg.de; isaac.moses@stud.th-deg.de

Abstract—Dynamic obstacle-aware navigation requires a mobile robot to preserve progress toward a goal while reacting to time-varying local hazards. This work presents a controlled comparative study of Deep Q-Network (DQN), Proximal Policy Optimization (PPO), and Advantage Actor-Critic (A2C) as learned local controllers for a TurtleBot3 Burger. Tabular Q-learning is included as a classical baseline. A saved SLAM map and the Nav2 global planner provide a fixed reference path, while a custom Gymnasium-compatible ROS 2 environment constructs a 32-dimensional observation from 24 normalized LiDAR bins and eight path/navigation features. The deep policies share the same seven discrete velocity actions, reward formulation, dynamic Gazebo benchmark, 700-step episode limit, and 140,000-timestep training budget. In the final 20-episode simulation evaluation, A2C achieved the highest success rate (75%), DQN achieved the lowest collision rate (10%) and mean cross-track error (0.07 m) but timed out in 40% of episodes, PPO achieved 35% success with a 65% collision rate, and Q-learning achieved 5% success. The trained deep policies were subsequently deployed without weight retraining on a physical TurtleBot3 using LDS-02 LiDAR, AMCL/TF localization, a saved real-room path, and an independent safety layer. A2C and PPO completed recorded forward-return trials, with PPO producing the fastest and most distance-efficient completed round trip. The contribution is a reproducible comparison and sim-to-real engineering workflow rather than a new RL algorithm.

Keywords—A2C, DQN, dynamic obstacle avoidance, Gymnasium, Nav2, PPO, reinforcement learning, ROS 2, sim-to-real, TurtleBot3.

I. INTRODUCTION

Autonomous mobile robots operating in warehouses, hospitals, service facilities, and inspection sites must move toward task goals while reacting to people, vehicles, carts, and other obstacles whose positions change over time. Classical navigation stacks remain highly effective for mapping, localization, global routing, and local collision avoidance. Nevertheless, their behaviour depends on model assumptions, cost functions, sensor quality, and manually tuned parameters. Reinforcement learning (RL) offers a complementary approach in which a local controller improves its action policy through repeated interaction with an environment [1], [2], [21].

Existing literature already demonstrates RL-based robot navigation; therefore, this project does not claim algorithmic novelty. Instead, it investigates an experimental question that is difficult to answer from published values alone: how do different RL families behave when the robot, observation, action set, reward, global path, obstacle scenario, training budget, and evaluation metrics are held as constant as possible? The main deep-RL comparison covers DQN, PPO, and A2C. Tabular Q-learning is retained as an interpretable classical baseline, but it uses a reduced discretized state and is therefore discussed separately when fairness is relevant.

The developed system uses ROS 2 Humble, Gazebo Classic, TurtleBot3 Burger, Nav2, Gymnasium, and Stable-Baselines3 (SB3). Nav2 produces a global path from a saved SLAM map. A learned policy acts as the local controller by publishing discrete

linear-angular velocity commands to `/cmd_vel`. After simulation evaluation, the trained deep models are transferred to a physical TurtleBot3 without weight updates. Real LiDAR and AMCL/TF replace simulated sensing and pose information, and a deterministic safety layer limits or overrides unsafe policy commands.

The remainder of this paper states the engineering problem, reviews the state of the art, identifies the research gap, describes the complete methodology, reports simulation and physical results, discusses three potential application areas, and documents the public GitHub repository required for reproducibility.

II. PROBLEM STATEMENT AND OBJECTIVES

A. Problem Statement

A global path alone does not determine how a differential-drive robot should react when local free space changes. Conversely, obstacle avoidance based only on current LiDAR clearance may prevent collisions but does not guarantee progress along the intended route. The problem addressed in this study is therefore to select short-horizon velocity actions that jointly preserve Nav2 path following, reduce collision risk in the presence of moving obstacles, and complete the navigation task within a finite horizon.

The principal research question is: *under identical ROS 2/Gazebo dynamic-navigation conditions, how do DQN, PPO, and A2C differ in success rate, collision rate, timeout behaviour, accumulated reward, and path-tracking accuracy?* A secondary question evaluates whether the simulation-trained deep policies can be executed on a physical TurtleBot3 without retraining by reconstructing a compatible observation from real sensors and localization.

B. Objectives

The objectives were to: 1) establish a repeatable dynamic TurtleBot3 benchmark with a fixed start, goal, saved global path, and scripted moving obstacles; 2) design a common observation, discrete action set, reward, and episode logic; 3) train DQN, PPO, and A2C with equal timestep budgets; 4) include tabular Q-learning as a classical reference; 5) evaluate each method over 20 final simulation episodes; 6) deploy the trained deep policies on a real TurtleBot3 without weight retraining; 7) apply a separate LiDAR safety layer; and 8) publish code, models, maps, paths, results, instructions, and demonstration links in a documented GitHub repository.

III. STATE OF THE ART

A. RL for Dynamic Mobile-Robot Navigation

RL-based path planning and obstacle avoidance have been investigated for both static and dynamic environments. A systematic review of 34 studies found growing use of deep RL but also highlighted long training times, reward sensitivity, heterogeneous environments, and limited real-world validation [1]. Earlier neural Q-learning work demonstrated static-and-dynamic obstacle navigation in virtual and physical

environments, but assumed global knowledge of robot, target, and obstacle states [2]. These results establish feasibility while also showing that reported performance depends strongly on what information the controller receives.

Dynamic obstacle navigation is more demanding than static avoidance because position alone does not encode motion intention. Raj and Kos argued that recent obstacle motion, velocity, and direction should be represented explicitly because two obstacles can occupy similar locations while moving differently [5]. The present project uses current LiDAR geometry and a dynamic-distance slot but does not estimate obstacle velocity; temporal obstacle modelling is therefore retained as future work.

B. Algorithm Selection

DQN extends Q-learning with a neural function approximator and is naturally suited to finite discrete actions [12]. LiDAR-based DQN navigation has been demonstrated in ROS/Gazebo, with recurrent memory and action skipping improving basic DQN behaviour [7]. DQN was therefore selected as the principal off-policy value-based method for the seven-action controller.

PPO is an on-policy actor-critic method that limits policy updates through a clipped surrogate objective [13]. PPO has been applied to LiDAR mobile-robot navigation, but published work emphasizes sensitivity to reward and network design. A recent TurtleBot3 study augmented PPO with LSTM memory and intrinsic curiosity to improve sparse-reward convergence and dynamic-obstacle performance in Gazebo and physical tests [6]. PPO was selected as the principal policy-gradient comparison to DQN.

A2C represents a direct actor-critic alternative derived from the asynchronous actor-critic family [14]. Prior A2C-based work translated laser and goal observations directly into movement commands and demonstrated physical transfer [9]. Including both PPO and A2C helps distinguish behaviour associated with actor-critic learning generally from behaviour associated with PPO specifically. Tabular Q-learning was included because improved Q-learning and dynamic-reward variants are still widely studied, yet the literature repeatedly reports convergence and scalability limitations in complex state spaces [3], [4].

TABLE I. LITERATURE-GROUNDED ALGORITHM SELECTION

Method	Learning family	Reason for inclusion
DQN	Off-policy value based	Matches seven discrete actions; LiDAR navigation precedent
PPO	On-policy actor-critic	Clipped policy update; widely used navigation baseline
A2C	On-policy actor-critic	Second actor-critic reference; prior real-robot transfer
Q-learning	Tabular value based	Classical interpretable baseline; exposes discretization limits

C. Hybrid Planning and Learned Local Control

The literature does not require learned navigation to replace the entire classical stack. Hierarchical systems have combined a traditional global planner with learned local trajectory replanning and velocity control [10], [22]. Map-based deep RL has also been transferred from simulation to real robots for dynamic local navigation [11]. This supports the architecture adopted here: SLAM and Nav2 provide global structure, whereas RL selects short-horizon local commands.

D. Sim-to-Real, Safety, and Benchmarking

Simulation is essential because thousands of exploratory interactions are unsafe and time-consuming on hardware. However, friction, latency, localization error, sensor noise, and imperfect resets create a sim-to-real gap. The review in [1] identifies the dominance of virtual evaluation and calls for standard environments and real validation. Learned policies also

lack formal safety guarantees; consequently, physical deployment commonly requires constrained commands, conservative speeds, or supervisory safety logic.

Published results are difficult to rank directly because studies differ in robots, observations, actions, rewards, obstacle motions, maps, training budgets, and success definitions. This motivates controlled in-project comparison rather than importing an external claim that one algorithm is universally best.

IV. RESEARCH GAP AND CONTRIBUTIONS

A. Research Gap

The reviewed studies establish that DQN, PPO, A2C, and modified Q-learning methods can support navigation. Nevertheless, no reviewed study exactly matches the combination used here: TurtleBot3 Burger; Nav2-generated fixed reference path; 24 LiDAR bins plus eight navigation features; seven discrete local actions; standard SB3 implementations of DQN, PPO, and A2C; a controlled dynamic-obstacle benchmark; a tabular Q-learning baseline; and subsequent physical inference using the same trained deep policies. The gap is therefore experimental and systems-oriented, not the absence of RL navigation research. The study additionally examines whether the ranking observed in simulation remains consistent after deployment on physical hardware.

B. Contributions

The work contributes: 1) a common ROS 2/Gazebo comparison environment; 2) a hybrid Nav2-global/RL-local architecture; 3) a shared 32-dimensional observation and seven-action interface for the deep models; 4) final outcome and path-tracking metrics over 20 simulation episodes per method; 5) a real TurtleBot3 inference pipeline with AMCL/TF localization and an independent LiDAR safety layer; and 6) a reproducible public repository containing the ROS 2 package, scripts, models, maps, paths, result summaries, documentation, and video links.

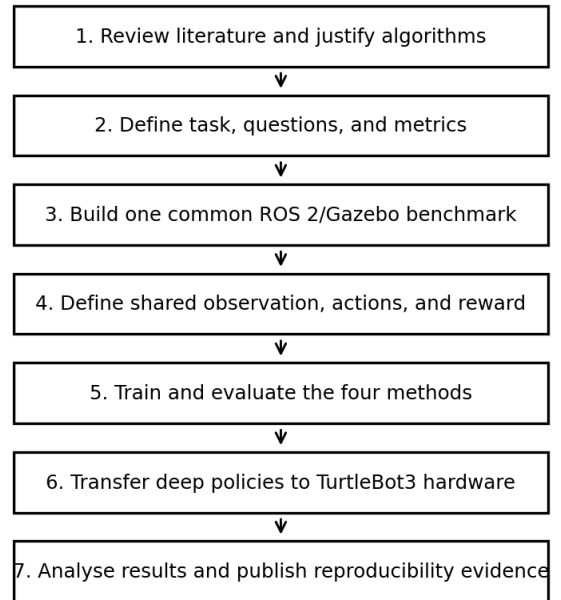


Fig. 1. Overall methodology of the completed comparative and sim-to-real study.

V. METHODOLOGY

A. System Architecture

The system separates global route generation from local control. A saved occupancy map produced with SLAM Toolbox is loaded [19], Nav2 computes a global path, and the path is

stored as a sequence of poses. At each control step, a custom Python node reads LiDAR and robot pose, calculates path-relative features, forms the observation vector, obtains an action from the RL model, and publishes the corresponding Twist command. In Gazebo, the policy command is sent directly to the simulated robot. On hardware, it first passes through the independent safety layer.

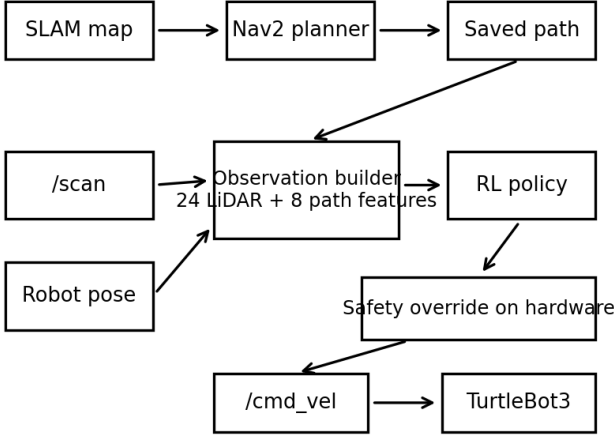


Fig. 2. Hybrid navigation architecture used in simulation and on the physical TurtleBot3.

ROS 2 topics include `/scan`, `/odom`, `/tf`, `/tf_static`, `/cmd_vel`, and `/amcl_pose`. Nav2 provides the path-computation action, while Gazebo reset services support episodic training. ROS 2 lifecycle management and LiDAR-compatible quality-of-service settings were required for reliable integration [17], [18].

B. Gymnasium Environment and Observation

Gymnasium provides the standard environment interface; it does not create the features or learn the policy. The custom Python environment defines `observation_space`, `action_space`, `reset()`, and `step()`. SB3 receives the numerical observation and reward returned by this environment and performs the algorithm-specific learning updates [15], [16]. During real-robot testing, a standalone runner reconstructs the same input order and calls `model.predict()` for deterministic inference; it does not train or update the network.

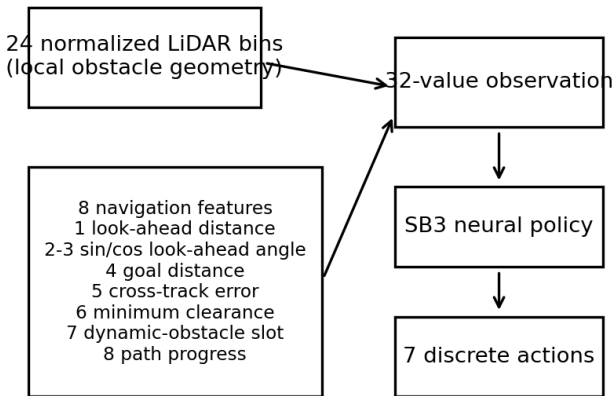


Fig. 3. Common deep-policy observation and action interface.

The first 24 values are normalized LiDAR bins distributed around the scan. The remaining eight values are: normalized look-ahead distance; sine and cosine of the look-ahead angle; normalized final-goal distance; normalized cross-track error (CTE); normalized minimum LiDAR clearance; a dynamic-obstacle-distance slot; and normalized progress along the saved

path. The look-ahead features express local route direction, CTE measures deviation from the reference path, and path progress distinguishes correct route advancement from merely becoming geometrically closer to the goal.

In simulation, the dynamic-obstacle slot could be derived from Gazebo model information. Exact obstacle identity and centre position were unavailable from the physical LiDAR. The real runner therefore retained the required input position but used an approximation during development and a neutral value in the final policy runner. This preserves tensor compatibility but constitutes a sim-to-real limitation.

C. Action Space

All deep policies used seven discrete velocity commands. The ordered pairs below denote (linear velocity in m/s, angular velocity in rad/s): forward (0.22, 0.00), forward-left (0.16, 0.75), forward-right (0.16, -0.75), turn-left (0.04, 1.20), turn-right (0.04, -1.20), slow-forward (0.08, 0.00), and stop (0.00, 0.00). The shared action set enables direct comparison of the deep algorithms and makes DQN applicable without reformulating the task as continuous control.

D. Reward and Episode Logic

The reward combines path progress, reduction in goal distance, heading alignment, cross-track error, time cost, and action/safety shaping. A compact representation is

$$r_t = 18 \max(\Delta s_t, 0) + 8 \Delta d_t + 0.10 \cos(e\psi_t) - 1.20 e_{CTE,t} - 0.03 + r_{action,t} + r_{safe,t} \quad (1)$$

where Δs_t denotes positive progress along the path, Δd_t denotes reduction in goal distance, $e\psi_t$ is the heading error to the look-ahead point, and $e_{CTE,t}$ is the cross-track error. Additional terms penalize unnecessary turning, unjustified stopping, and dangerous proximity, while rewarding safe aligned forward motion. Terminal rewards were +400 for goal completion, -400 for collision, -180 for excessive off-path deviation, and -80 for timeout.

An episode terminated when the robot reached the goal, collided, moved excessively off-path, or triggered the stuck logic. It was truncated at 700 steps. This distinction follows the modern Gymnasium interface, which separates natural task termination from external limits [16].

E. Algorithms and Training Protocol

TABLE II. PRINCIPAL TRAINING SETTINGS

Method	Key settings	Budget
DQN	lr 1e-4; buffer 60k; batch 64; gamma 0.99; target 500	140k steps
PPO	lr 3e-4; n_steps 256; batch 64; gamma 0.99; clip 0.20	140k steps
A2C	lr 7e-4; n_steps 32; gamma 0.99; GAE 0.95	140k steps
Q-learn.	alpha 0.15; gamma 0.98; epsilon 1.0 to 0.05	200 episodes

The deep models were trained with Python 3.10.12, SB3 2.3.2, Gymnasium 0.29.1, PyTorch 2.2.2+cpu, and NumPy 1.26.4. Each received 140,000 interaction timesteps. Because success, collision, off-path, and stuck conditions ended episodes early, the number of completed training episodes differed among algorithms. Dividing 140,000 by the 700-step maximum would yield 200 episodes only if every episode reached its maximum length. Q-learning was instead trained for exactly 200 manually controlled episodes using a five-dimensional discretized state and a seven-action Q-table.

Final evaluation used 20 episodes per method under the same saved simulation path and dynamic-obstacle scenario. The primary metrics were success, collision, timeout, off-path outcome, accumulated reward, final goal distance, CTE, and path progress. The study did not fix or publish random seeds; the

20-run results are therefore descriptive rather than confidence-bounded statistical estimates.

F. Evaluation Metrics

Success, collision, timeout, and off-path rates were calculated from the final 20 evaluation episodes. Cross-track error (CTE) was computed at each sample as the minimum Euclidean distance between the robot position and any pose on the saved path. The reported mean CTE is the arithmetic mean of the episode-level CTE values exported by the evaluation script. For completed hardware runs, path efficiency was defined as $\eta = \text{planned path length} / \text{actual travelled distance}$. An efficiency of 1 denotes travel equal to the planned path length, whereas lower values indicate additional motion.

VI. EXPERIMENTAL SETUP

A. Simulation Benchmark

The simulation used TurtleBot3 Burger in Gazebo Classic with ROS 2 Humble. The fixed saved path contained 180 poses and had a length of 4.52 m. The start pose was (0.7376, -1.0048, 3.1387), and the goal pose was (-2.0694, 1.7654, 0.0). Two scripted TurtleBot3 entities acted as moving obstacles. The control interval was 0.10 s, the look-ahead distance was 0.60 m, the goal threshold was 0.30 m, and the maximum episode length was 700 steps.

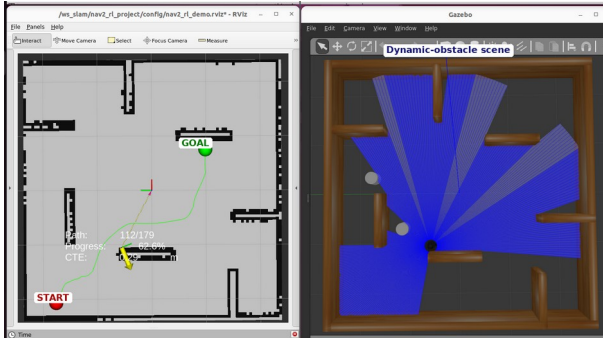


Fig. 4. Controlled simulation benchmark: RViz path-tracking view (left) and Gazebo scene with the TurtleBot3 and two scripted moving obstacles (right).

B. Physical TurtleBot3 Setup

The physical platform was a TurtleBot3 Burger with OpenCR motor control, an LDS-02 LiDAR, wheel odometry, IMU, and a Raspberry Pi running Ubuntu 22.04 and ROS 2 Humble. Robot bring-up ran on the Raspberry Pi, while localization, path processing, policy inference, safety logic, and experiment recording ran in a Docker-based laptop environment over a shared ROS 2 network domain.

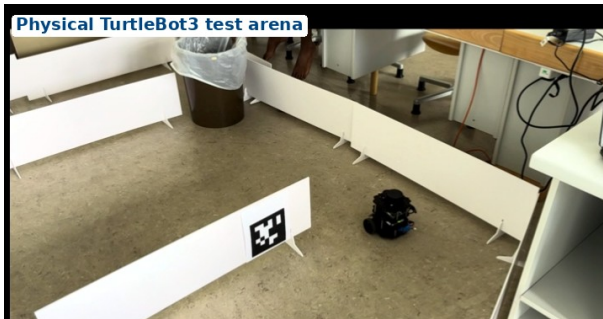


Fig. 5. Physical TurtleBot3 Burger in the indoor arena used for real-policy inference and safety-layer experiments.

The real occupancy map used a resolution of 0.05 m/pixel, meaning that each map pixel represents a 5 cm by 5 cm floor region. AMCL, based on adaptive particle-filter localization [20], and TF provided the map-to-base_footprint pose. The

forward global path contained 132 poses over 3.351 m, and the return path used the reversed pose sequence. The real test runner supported A2C, DQN, and PPO model archives; tabular Q-learning was not deployed.

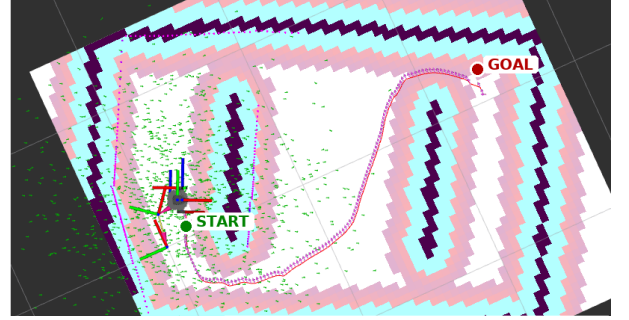


Fig. 6. Real-room RViz map and saved 132-pose path (3.351 m), with the common start and goal used for forward and return trials.

C. Independent Hardware Safety Layer

The physical system is a hybrid RL-plus-safety controller. The learned model proposes an action, but deterministic LiDAR rules may reduce or replace it. Slowdown activates below 0.55 m, recovery rotation below 0.28 m, and emergency all-stop at or below 0.14 m. Linear and angular commands are limited to 0.22 m/s and 1.20 rad/s, respectively. A run is also stopped if CTE exceeds 0.85 m. Recovery selects the clearer left or right sector and rotates in place until clearance is restored.

TABLE III. REAL-ROBOT SAFETY PARAMETERS

Parameter	Value	Purpose
Slowdown distance	0.55 m	Reduce forward speed
Recovery distance	0.28 m	Rotate toward clearer side
Emergency distance	0.14 m	Publish zero velocity
Maximum CTE	0.85 m	Stop excessive path deviation
Max. linear/angular	0.22 m/s; 1.20 rad/s	Constrain physical commands

VII. RESULTS

A. Final Simulation Evaluation

TABLE IV. OUTCOME RATES OVER 20 FINAL SIMULATION EPISODES

Method	Success	Collision	Timeout	Off-path
A2C	75%	25%	0%	0%
DQN	50%	10%	40%	0%
PPO	35%	65%	0%	0%
Q-learn.	5%	60%	30%	5%

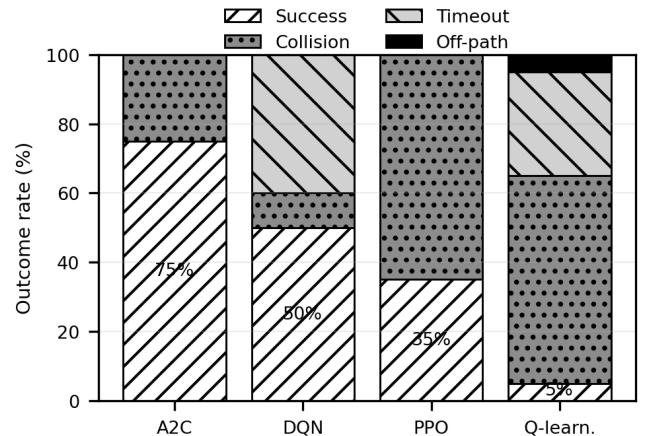


Fig. 7. Distribution of final simulation outcomes. The Q-learning percentages include the previously omitted 5% off-path outcome.

TABLE V. SIMULATION TRACKING AND EFFICIENCY METRICS

Method	Mean reward	Mean steps	Goal dist.	Mean CTE	Progress
A2C	178.99	198.45	0.61 m	0.27 m	0.89
DQN	121.83	476.95	0.61 m	0.07 m	0.86
PPO	-254.81	235.90	1.06 m	0.34 m	0.79
Q-learn.	-601.90	438.85	2.14 m	0.45 m	0.50

A2C reached the goal in 15 of 20 episodes and achieved the highest success rate. DQN completed 10 episodes, collided in only two, and obtained the lowest reported mean episode-level CTE, indicating the closest average path adherence among the evaluated methods; however, eight episodes reached the 700-step limit. PPO completed seven episodes but collided in 13, showing active but aggressive behaviour in this reward/task configuration. Q-learning completed only one episode, with 12 collisions, six timeouts, and one off-path termination. The result supports the literature-based expectation that a small discretized table is inadequate for a continuous, high-dimensional dynamic navigation problem.

These outcomes do not establish a universal ranking. They show that the same benchmark exposes different failure modes: A2C favoured completion, DQN favoured path adherence but frequently stalled or stopped, and PPO favoured action continuity but accepted excessive collision risk under the learned policy.

B. Physical Robot Results

TABLE VI. RECORDED FULL-SPEED HARDWARE OUTCOMES

Method	Forward	Return	Representative measured outcome
DQN	Success once	Failed	109.14 s forward; 4.44 m; mean CTE 0.125 m
A2C	Success	Success	161.74 s round trip; 14.02 m; eta 0.478
PPO	Success	Success	126.68 s round trip; 9.11 m; eta 0.736

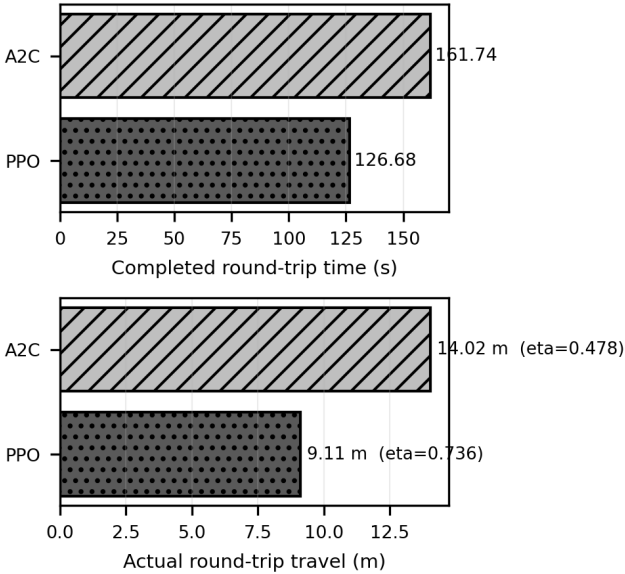


Fig. 8. Comparison of the completed A2C and PPO hardware round trips. Efficiency eta is planned distance divided by actual travel distance.

DQN completed one forward trial with good path precision but failed to complete the return and later repeated trials were inconsistent. A2C completed both directions, but its forward run contained substantial oscillation and unnecessary travel. PPO completed both directions in the lowest recorded time and with the least actual travel among the completed round trips. The goal threshold of 0.30 m explains why successful runs can end before path progress reaches exactly 1.0.

The hardware dataset is small and cannot support population-level success rates or statistical claims that PPO is generally superior. The results should be interpreted as proof-of-concept

observations. The experiment logs recorded no LiDAR collision-proxy event during the successful full-speed runs, but this proxy is not equivalent to physical-contact ground truth. Moreover, many policy actions were modified by slowdown or recovery logic; the measured behaviour represents the combined learned policy and deterministic safety system.

VIII. DISCUSSION

The controlled simulation comparison shows why state-of-the-art findings cannot be transferred uncritically between projects. A2C, DQN, and PPO used the same information and actions, yet converged to distinct behaviours because their update mechanisms interact differently with reward shaping and episode termination. DQN's low CTE did not imply reliable task completion, while PPO's absence of timeouts did not imply safe navigation. A2C achieved the highest success rate but did not produce the most efficient physical motion.

Simulation performance also did not predict hardware ranking directly. Localization variation, LiDAR noise, command latency, traction, battery state, and the simulation-specific dynamic-obstacle slot changed the effective observation distribution. The independent safety layer was necessary for cautious testing but also altered the policy's command sequence. Consequently, this study supports a systems perspective: real RL navigation performance depends not only on the learned network, but also on localization, path construction, command ownership, sensor timing, and safety supervision.

The hybrid architecture proved useful because it assigned different roles to mature and learned components. SLAM and AMCL supplied spatial consistency, Nav2 supplied route intention, and RL supplied local action selection. This avoided requiring the policy to discover a global route while still enabling a dynamic response. Future native integration as a Nav2 controller plugin would allow better lifecycle management, command arbitration, and benchmarking against conventional Nav2 controllers.

IX. POTENTIAL APPLICATION AREAS

A. Intralogistics and Flexible Manufacturing

Warehouses and factories contain fixed aisle structure together with people, forklifts, carts, pallets, and temporary blockages. A Nav2 global route can encode delivery intent, while a learned local controller can select short-horizon motion around transient obstacles. The approach could support material transport, line-side replenishment, and mobile manipulation bases after broader map, load, and safety validation.

B. Hospitals and Public-Service Robotics

Hospital corridors, airports, hotels, and public buildings require service robots to navigate around pedestrians and movable equipment while maintaining a destination route. The combined path-following and local obstacle representation is relevant to delivery, guidance, and cleaning robots. Deployment in human spaces would require substantially more real trials, formal speed and separation constraints, contact sensing, and verified fail-safe stopping.

C. Inspection and Emergency-Support Robotics

Industrial plants, utility rooms, construction sites, and post-incident facilities frequently contain temporary or moving obstructions. A robot could follow an inspection route while reacting to changed access conditions. The current work provides a basis for such systems, but future versions should include multi-map training, temporal obstacle prediction, robust localization under degraded visibility, and explicit uncertainty handling.

X. LIMITATIONS AND FUTURE WORK

The main limitations are: one principal simulation route; only 20 final evaluation episodes per method; no fixed random-seed protocol or confidence intervals; a limited number of hardware trials; no physical Q-learning deployment; no exact reconstruction of the simulation dynamic-obstacle feature; no explicit obstacle-velocity or temporal scan history; no domain randomization; no quantitative comparison against a conventional Nav2 local controller; and no formal safety guarantee. Q-learning also did not share the deep models' full observation or timestep-defined budget and must therefore be treated as a contextual rather than strictly equal baseline.

Future work should evaluate multiple maps, start-goal pairs, obstacle speeds, and irregular trajectories; repeat each condition over fixed seeds; report confidence intervals; incorporate stacked scans, tracking, recurrent memory, or velocity estimates; extend training beyond 140,000 timesteps with tuned rewards and curriculum learning; randomize sensor noise, friction, latency, and dynamics; perform more controlled physical trials; add bumper/contact and hardware emergency-stop devices; compare against standard Nav2 local controllers; and implement the learned policy as a Nav2 controller plugin.

XI. REPRODUCIBILITY AND GITHUB REPOSITORY

The complete project is maintained as a public ROS 2 Python package. The repository contains the simulation and hardware scripts, package metadata, configuration files, maps, saved forward/return paths, trained DQN/PPO/A2C models, Q-table, final result summaries, technical documentation, and demonstration-video links. Raw ROS bags and large videos are stored externally to avoid unnecessary Git history growth.

Repository: <https://github.com/mathiyalagan-vasantharaj/ros2-rl-smart-navigation-turtlebot3>

The README provides the prerequisites, environment variables, build commands, launch order, simulation training and evaluation steps, real TurtleBot3 bring-up, localization and path-generation workflow, experiment commands, emergency stopping procedure, and expected outputs. Reproducibility is additionally supported by a version lock, data dictionary, interface description, safety documentation, simulation-to-real workflow, and consolidated CSV summaries.

XII. CONCLUSION

This project completed a controlled comparison of DQN, PPO, and A2C for discrete local navigation of a TurtleBot3 following a Nav2 global path in a dynamic Gazebo environment, with tabular Q-learning as a classical baseline. A2C achieved the highest simulation success rate, DQN achieved the lowest collision rate and reported episode-level CTE but experienced frequent timeouts, PPO showed aggressive simulated behaviour, and Q-learning was insufficient for the continuous dynamic task. The deep policies were transferred without retraining to a physical TurtleBot3 using real LiDAR, AMCL/TF localization, and a common saved path. A2C and PPO completed recorded round trips, with PPO producing the best measured time and distance among the completed trials. The study's principal value is the reproducible comparison and end-to-end sim-to-real workflow. It also demonstrates that successful physical RL navigation depends on classical localization, global planning, conservative command limits, and independent safety supervision in addition to the learned policy.

ACKNOWLEDGMENT

The authors thank the course teaching team for guidance and access to the TurtleBot3 platform.

REFERENCES

- [1] K. Almazrouei, I. Kamel, and T. Rabie, "Dynamic obstacle avoidance and path planning through reinforcement learning," *Applied Sciences*, vol. 13, no. 14, Art. no. 8174, 2023, doi: 10.3390/app13148174.
- [2] M. Duguleana and G. Mogan, "Neural networks based reinforcement learning for mobile robots obstacle avoidance," *Expert Systems with Applications*, vol. 62, pp. 104–115, 2016, doi: 10.1016/j.eswa.2016.06.021.
- [3] C. Wang, X. Yang, and H. Li, "Improved Q-learning applied to dynamic obstacle avoidance and path planning," *IEEE Access*, vol. 10, 2022, doi: 10.1109/ACCESS.2022.3203072.
- [4] A. Gharbi, "A dynamic reward-enhanced Q-learning approach for efficient path planning and obstacle avoidance in mobile robotics," *Applied Computing and Informatics*, 2024, doi: 10.1108/ACI-10-2023-0089.
- [5] R. Raj and A. Kos, "Dynamic obstacle avoidance technique for mobile robot navigation using deep reinforcement learning," *International Journal of Emerging Trends in Engineering Research*, vol. 11, no. 9, pp. 307–314, 2023.
- [6] Q. Zhang et al., "Path planning of mobile robot in dynamic obstacle avoidance environment based on deep reinforcement learning," *IEEE Access*, vol. 12, pp. 189136–189150, 2024, doi: 10.1109/ACCESS.2024.3507016.
- [7] I. Kim, S. H. Nengroo, and D. Har, "Reinforcement learning for navigation of mobile robot with LiDAR," in *Proc. 5th Int. Conf. Electronics, Communication and Aerospace Technology (ICECA)*, 2021, pp. 148–154, doi: 10.1109/ICECA52323.2021.9675949.
- [8] H. Taheri, S. R. Hosseini, and M. A. Nekoui, "Deep reinforcement learning with enhanced PPO for safe mobile robot navigation," *arXiv:2405.16266*, 2024.
- [9] M. Dobrevski and D. Skočaj, "Deep reinforcement learning for map-less goal-driven robot navigation," *International Journal of Advanced Robotic Systems*, vol. 18, no. 1, 2021, doi: 10.1177/1729881421992621.
- [10] A. Pokle et al., "Deep local trajectory replanning and control for robot navigation," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2019, doi: 10.1109/ICRA.2019.8794062.
- [11] G. Chen et al., "Robot navigation with map-based deep reinforcement learning," in *Proc. IEEE Int. Conf. Networking, Sensing and Control (ICNSC)*, 2020, doi: 10.1109/ICNSC48988.2020.9238090.
- [12] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015, doi: 10.1038/nature14236.
- [13] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv:1707.06347*, 2017.
- [14] V. Mnih et al., "Asynchronous methods for deep reinforcement learning," in *Proc. 33rd Int. Conf. Machine Learning, PMLR*, vol. 48, pp. 1928–1937, 2016.
- [15] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.
- [16] M. Towers et al., "Gymnasium: A standard interface for reinforcement learning environments," *arXiv:2407.17032*, 2024.
- [17] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, Art. no. eabm6074, 2022, doi: 10.1126/scirobotics.abm6074.
- [18] S. Macenski, F. Martín, R. White, and J. Ginés Clavero, "The Marathon 2: A navigation system," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2020.
- [19] S. Macenski and I. Jambrecic, "SLAM Toolbox: SLAM for the dynamic world," *Journal of Open Source Software*, vol. 6, no. 61, Art. no. 2783, 2021, doi: 10.21105/joss.02783.
- [20] D. Fox, "KLD-sampling: Adaptive particle filters," in *Advances in Neural Information Processing Systems 14*, 2001.
- [21] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [22] B. Wang, Z. Liu, Q. Li, and A. Prorok, "Mobile robot path planning in dynamic environments through globally guided reinforcement learning," *IEEE Robotics and Automation Letters*, vol. 5, no. 4, pp. 6932–6939, 2020, doi: 10.1109/LRA.2020.3026638.